SigmaStorm: Bounded-State Sequence Modeling without Dense Token-to-Token Attention

A technical report on mechanisms, falsification gates, and prototype-scale evidence

Ravinder Singh
https://github.com/sainibhaowal/NOESIS-Sigma

November 2026

Abstract

We describe **SigmaStorm**, a sequence model that replaces dense token-to-token attention with a fixed-size recurrent state advanced by a bounded transition law. Each token is processed by heterogeneous operators—a short convolution, a gated delta-rule associative memory with a multiscale decay spectrum, per-token-gated slot memory, a ring of chunk summaries, and optional window attention—whose proposals are mixed by independent sigmoid gates, contracted by an energy gradient, and projected onto a norm ball. Every piece of carried state has fixed configured capacity, so state-tensor memory does not grow with context. The implementation is tested for chunk equivalence at a split inside a slot-consolidation and a summary block; the reported maximum hidden-state difference is 1.6×10^{-6} for that configuration. Equivalence gates measure the delta rule against its sequential reference to 4.2×10^{-7}. On synthetic recall the model reaches 0.974 on multi-query associative recall against 0.062 chance; the decay spectrum retains long-range recall in the tested frozen-rate task (1.000 vs. 0.297 at distances ≥ 256 tokens). We report a gated slot read that makes the slot memory content-addressed on a frozen-embedding presence task: recall rises from near chance (0.56/0.54/0.56) to **0.977/1.000/0.953** across three seeds, with a +2.26-nat memory-ablation delta. The small paired language-model check changed validation CE from 3.0760 to 3.0826 nats; this is a single no-regression check, not evidence of an improvement. A 21M-parameter TinyStories model trained on 348M tokens produces the recorded held-out loss 1.2523 (PPL 3.498) and locally coherent on-genre samples. We also report what is *not* established: a capacity benefit from state expansion, a reliable benefit from chunk summaries, long-context training at 256K tokens, and any head-to-head language-modeling comparison with a transformer. This is a prototype evidence record, not a frontier claim.

1 Introduction

Transformers owe much of their success to exact token-to-token attention, and much of their cost to it as well: the pairwise attention matrix is quadratic in sequence length, while the autoregressive key–value cache grows linearly with the prefix. Long-context inference therefore faces both a quadratic interaction cost and a growing cache [Vaswani et al., 2017, Dao et al., 2022]. Recent frontier systems attack this cost with compressed or sparse attention and with aggressive KV-cache compression [Kimi Team, 2026, DeepSeek-AI, 2026]. A complementary question is architectural: can a model replace the pairwise interaction matrix with a *fixed-size* recurrent state that nevertheless supports several distinct kinds of memory?

NOESIS-Σ is the research project name. In this project, NOESIS is also used as the adopted descriptive expansion **N**eural **O**perator **E**nergy-guided **S**tateful **I**nference **S**ystem; this expansion is a project convention, not the historical meaning of the Greek word. SigmaStorm is the executable model and Python package developed under that project.

SigmaStorm is a concrete answer in code and measurement. One block advances a token-local latent once:

$$\begin{aligned} u &= \mathrm{RMSNorm}(x), \\ d_i &= \mathrm{op}_i(u) \quad \text{for each enabled } i, \\ m &= \frac{\sum_i \alpha_i(u)\, d_i}{\sum_i \alpha_i(u)}, \\ \alpha_i(u) &= \sigma(w_i^\top u), \\ v &= \Pi_R\Big(u + \Delta t(u)\,\big[m - \nabla E(u) + u\big]\Big), \\ x' &= x + W_o v. \end{aligned} \tag{1}$$

The gates α_i are independent sigmoids rather than a softmax; each operator output is individually RMS-normalised before gating; the mixture is normalised by its gate sum so that opening more gates does not amplify the proposal; and the projection enforces a hard norm bound. Depth plays the role of transition time, while recurrent state—not the token prefix—carries information across positions.

The operators are chosen to cover different failure modes of sequence modeling: a short convolution for local order, a gated delta rule for key→value associations, a slot memory for long-horizon prototypes, a fixed ring of pooled chunk summaries for a bounded global path, optional window attention for precise local copying, and an energy term that stabilises the latent trajectory. Table 1 lists their cost and state. Every carried state tensor has a shape independent of T for a fixed configuration; this is a state-capacity property, not evidence that the model has learned 256K-token behavior.

Scope and evidence policy. This report is deliberately a prototype record. It separates three things that are often conflated in architecture papers: (i) mechanisms that are *implemented*, (ii) properties that are *measured* by

a falsification gate, and (iii) capabilities that are *demonstrated* by a controlled experiment. Appendix C records earlier claims that failed to reproduce. The central negative results are reported with the same prominence as the positive ones.

1.1 Contributions

1. **An executable transition law with a hard state contract.** Section 3 gives the operator mixture, energy contraction, bounded projection, and the carried-state rules. The same code runs a 2M-parameter mechanism model and a 21M-parameter language model.
2. **A carried-state chunking contract.** The delta rule, slot memory (including its write accumulator), summary ring (including its partial-chunk accumulator), and window ring are carried in the state. A gate splits a sequence at position 50—inside a 64-token slot block and a 32-token summary block—and matches the full forward to 1.6×10^{-6} for the tested configuration (Table 4).
3. **A multiscale decay spectrum.** Each delta-rule head starts at its own log-spaced rate. With the spectrum frozen, recall at distances ≥ 256 is 1.000, versus 0.297 for a single frozen rate (Section 6.2).
4. **A gated, content-addressed slot read.** A novelty dead-zone on writes, a separate read temperature, and a learned match-confidence gate turn a diffuse softmax average into an address. Presence recall moves from chance to 0.977/1.000/0.953 over three seeds, and zeroing the memory now costs +2.26 nats instead of +0.15 (Section 6.4).
5. **No-regression checks with mixed outcomes reported.** The gated read leaves MQAR close to the previous arm (0.990/0.987 vs. 0.988/0.979). Aggregate recall and the single paired language-model check are reported without claiming an improvement; one aggregate seed is lower and validation CE changes from 3.0760 to 3.0826 nats.
6. **An honest capability map.** A 21M model produces locally coherent, on-genre TinyStories samples (held-out PPL 3.498), while state expansion, chunk-summary benefit, and 256K-context training remain unproven at this scale (Table 3).

1.2 Claims and non-claims

We claim: the mechanisms above are correct under their equivalence tests; the decay spectrum and the gated slot read have controlled evidence on synthetic recall; a small language model trains and generates; state size is independent of context.

We do not claim: that SigmaStorm matches or beats transformers at language modeling, that 256K-token context has been trained or evaluated, that state expansion increases capacity, that chunk summaries reliably improve quality, or that any result here extrapolates to frontier scale. The largest model in this report is 21M parameters, trained on 348M tokens of a synthetic children's corpus; the timing measurements are single-machine CPU forward passes, not production throughput.

2 Related work

Efficient attention and long context. Exact attention is quadratic in sequence length and its KV cache grows linearly with the prefix. Sparse and windowed attention reduce the interaction graph [Beltagy et al., 2020, Zaheer et al., 2020]; IO-aware kernels reduce the constant [Dao et al., 2022]. Frontier long-context systems now combine sparse attention with compressed caches, reporting hundreds of bytes per token [DeepSeek-AI, 2026, Kimi Team, 2026]. SigmaStorm does not approximate dense attention: it removes the pairwise matrix and replaces it with a recurrent state plus a small number of bounded memory paths. Window attention, when enabled, is a sliding local operator with a ring-buffer state, not a global mask.

Linear recurrences and delta rules. Linear attention rewrites softmax attention as a running key–value summary [Katharopoulos et al., 2020] and is equivalent to fast-weight programming [Schlag et al., 2021]. State-space models provide long-range convolution and selective recurrences [Gu et al., 2022, Gu and Dao, 2024]; RetNet and RWKV interleave decayed recurrence with token-wise gating [Sun et al., 2023, Peng et al., 2023]; gated linear attention adds hardware-efficient chunkwise training [Yang et al., 2024]. The delta rule used here belongs to the error-correcting family: the state is rewritten by the residual between an observed value and the value the state currently predicts, which is what makes a binding overwrite rather than blur [Yang et al., 2025]. SigmaStorm contributes a multiscale initialisation of that rule and a chunkwise implementation with exact carry.

Structured memory. External-memory networks and differentiable stores introduced explicit read/write addressing [Graves et al., 2014]; modern Hopfield networks interpret attention-like reads as energy-minimising retrieval [Ramsauer et al., 2021]. SigmaStorm keeps several memory forms with different time scales—fast associative state, consolidated prototypes, pooled summaries—and exposes them through independent gates. The energy term in the transition is a *contraction*, not a multi-basin attractor: strong convexity forbids many equilibria, and the memory operators, not the energy, hold information. This distinction is deliberate and is revisited in Section 3.3.

Recall benchmarks. Multi-query associative recall (MQAR) isolates the ability to bind and retrieve key–value pairs and has become a standard diagnostic for efficient architectures [Arora et al., 2023]. Mechanistic work on induction heads identifies the same copy-and-match primitive inside transformers [Olsson et al., 2022]. We use MQAR for associative recall, a delayed-recall task with distance buckets for the decay spectrum, an aggregate count task for the summary path, and a frozen-embedding presence task for the slot memory. Keeping the tasks separate matters: one operator can otherwise hide another's defect.

Small language models. TinyStories showed that models with a few million parameters can produce coherent English when the corpus is syntactically simple [Eldan and Li, 2023]. It is the right scale for a prototype: a full training run fits on two 16 GB T4 GPUs in two short sessions, and the generated text is easy to inspect. It is not evidence of general language ability, and perplexities are comparable only within the same tokenizer and split.

Positioning. SigmaStorm is not a transformer with layers removed, and it is not a single linear-attention approximation. Its claim is compositional: several operators with different inductive biases, mixed by independent gates on a bounded latent, with a strict state-size contract. The report tests the pieces separately and reports where the composition has not yet paid off.

3 The Sigma transition law

3.1 The block

A sequence is processed as a stream. For each token position t the model keeps a residual stream $x_t \in \mathbb{R}^d$ and a fixed-size recurrent state S_t. One block applies Equation 1 once: normalise, offer the normalised latent to the enabled operators, mix their proposals, subtract an energy gradient, and project. Stacking L blocks applies the law L times to the token-local latent, while S_t carries information across tokens.

Three details are not cosmetic.

- **Independent sigmoid gates.** A softmax over operators forces them to compete for a fixed total and can suppress an operator that is useful on a particular token. Independent sigmoids let any subset contribute simultaneously; they are mixture weights, not a compute skipping mechanism.
- **Convex normalisation.** Dividing by the gate sum keeps the proposal scale from growing merely because more operators are enabled. The normalisation is a stability device, not evidence that any particular routing distribution is optimal.
- **Per-operator RMSNorm.** Normalising each proposal before gating puts heterogeneous operator outputs on a comparable scale, so the gate is not dominated by an arbitrary raw output magnitude.

The step size is $\Delta t(u) = \Delta t\,(0.5 + \sigma(a^\top u))$ with $\Delta t = 0.5$ by default. The projection acts on the latent v only, never on the residual stream; this is what keeps token identity available to the associative operator.

3.2 Operator family

Table 1 lists the operators, their role, intended fixed-configuration cost, and state size. Window attention is enabled on one block in four by default and can be removed entirely; it retains a bounded ring history. The current implementation also materialises a mask over each input chunk, so the strict bounded-window runtime expression below should not be read as a guarantee for arbitrarily large unchunked calls.

3.3 Energy contraction

The potential is

$$\begin{aligned} E(u) &= \tfrac{1}{2}u^\top Du + \tfrac{1}{2}\|L^\top u\|^2, \\ -\nabla E(u) &= -(Du + LL^\top u), \end{aligned} \tag{2}$$

with D diagonal positive and $L \in \mathbb{R}^{d\times r}$ low-rank. E is strongly convex, so $-\nabla E$ is an anisotropic contraction towards the origin. It is a stabiliser, not a multi-basin memory: strong convexity forbids many equilibria by construction. The slot memory and the delta rule are what hold information; the energy term keeps the latent from drifting and makes the projected dynamics well behaved. An earlier design that treated the latent as a multi-well attractor produced no usable memory and was discarded.

3.4 Bounded projection

The projection is $\Pi_R(z) = z \cdot \min(1, R/\|z\|)$ with $R = 16$. It is applied after the transition and before the output projection. Because the residual stream is additive ($x' = x + W_o v$), the bound constrains the *proposal*, not the accumulated representation; this distinction was one of the three fixes that took MQAR from chance to 0.97.

3.5 State contract and chunk invariance

Observation 1 (Carried-state chunking contract)**.** *For a fixed configuration, carrying the state S_t of Table 2 between contiguous chunks is intended to make the hidden representation independent of the caller's chunk boundaries. The current gate tests one split at position 50, inside both a 64-token slot block and a 32-token summary block; it reports a maximum hidden-state difference of 1.6×10^{-6}. This is an implementation check for that configuration, not an empirical claim over every possible boundary, length, or hardware backend.*

Why the contract is plausible. Every operator in Table 1 is implemented as a causal scan whose value at position t depends on its carried state and tokens up to t. The short convolution carries its $k-1$ token tail; the delta rule carries its matrices and end-of-chunk state; the slot memory carries the ring of prototypes *and* unconsolidated write accumulators; the summary ring carries completed summaries and its partial-chunk accumulator; and the window carries its ring buffer and length. The controller and energy are token-local. These are the design conditions needed for boundary independence. Gradient detaching changes the backward graph, not the forward state update.

3.6 Streaming forward and update lifecycle

3.7 Training interface

Training processes `micro_chunk` tokens per forward call (default 512 in the current training interface) and carries the state within a sequence. The differentiable horizon is `bptt_chunks` × `micro_chunk` (default $4 \times 512 = 2048$ tokens); the state is detached at the window boundary.

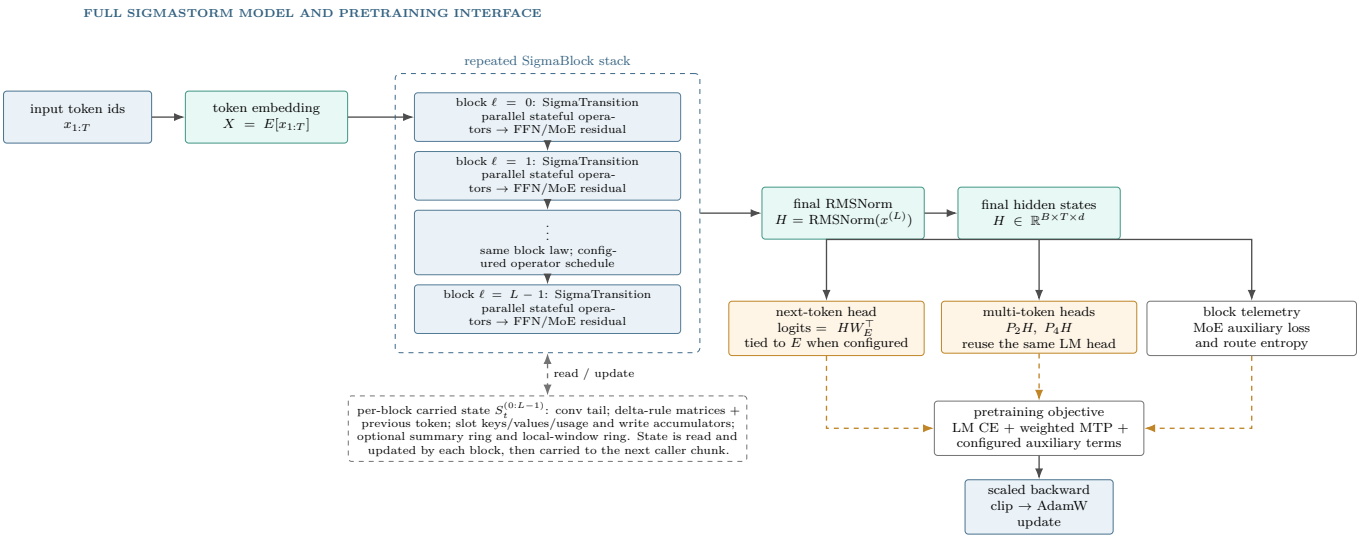


Figure 1: Top-level SigmaStorm architecture used for pretraining. Token ids are embedded and passed through L SigmaBlocks; each block applies the stateful SigmaTransition and then a state-free gated FFN or sparse-MoE residual. The block-local state is carried across caller chunks, while depth is the repeated transition stack. The final normalised hidden states feed the tied language-model head and the configured multi-token prediction projections; block telemetry contributes the auxiliary terms shown in the pretraining objective. The diagram is the implemented model interface, not an instruction-tuning, preference-optimisation, or frontier-scale system diagram.

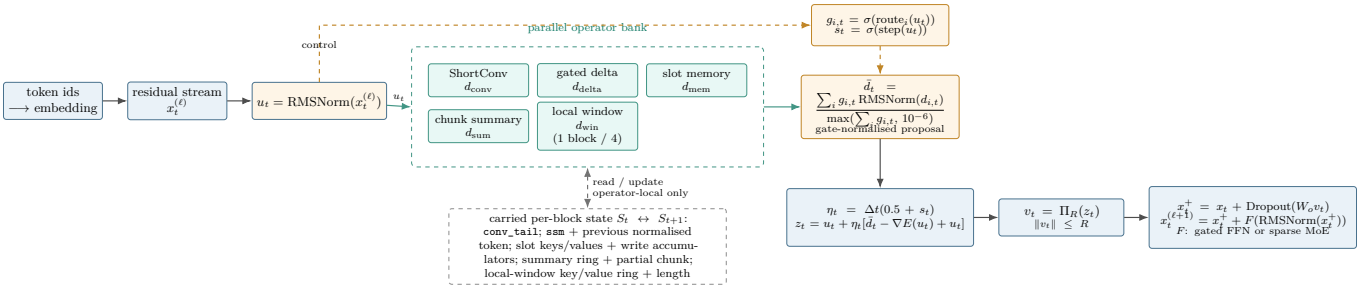


Figure 2: Implemented SigmaStorm block dataflow. The residual stream is normalised once and offered to the enabled operators in parallel; the operator cards are not sequential layers. Each operator reads and updates only its own fixed-size carried state. The controller emits independent sigmoid gates and a step signal. Per-operator RMS-normalised proposals are gate-normalised, combined with the learned energy contraction and the latent residual, projected onto the radius-R ball, and added back through W_o. The resulting transition is followed by the state-free gated FFN or sparse MoE residual. The state rail lists the actual fields carried across caller chunks; the figure is a dataflow contract, not a literal tensor-layout drawing.

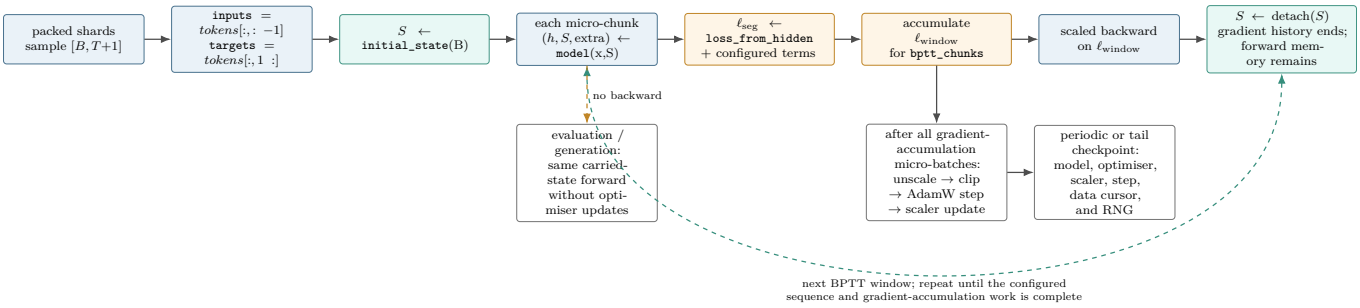


Figure 3: Implemented streaming training lifecycle. A sampled sequence receives one initial carried state, which is passed through every micro-chunk and every truncated-backpropagation window. The state is detached only after a window backward pass: this truncates gradient history but does not reset forward memory. Optimiser work occurs only after gradient accumulation; evaluation and generation use the same carried-state forward interface without an optimiser update.

One reported Kaggle training profile used 256-token micro-chunks, so the paper reports the configured value rather than treating one profile as a universal default. Forward context is not limited by this horizon, but gradients are.

Table 1: Operators in the SigmaStorm block. Costs are per sequence of length T; state is per sample and independent of T. k is the convolution kernel, H the number of delta heads, k_{exp} the state-expansion factor, $d_h = d/H$ the head width, S the number of summary slots, and W the window length.

Operator	Role	Cost	State
ShortConv	local order; the fast proposal	$O(Tkd)$	$k-1$ tokens
Gated delta rule	associative recall (key→value bindings)	linear in T for fixed chunks and width	Hk_{exp} matrices $d_h \times d_h$
Slot memory	long-horizon prototypes with per-token gated writes	$O(Tn_sd)$	$n_s \times d$ + accumulators
Chunk summaries	bounded global summary path without T^2	$O(TSd)$	ring of S summaries
Window attention	precise local copying (1 block in 4, optional)	bounded-window intent; chunk-mask caveat above	ring buffer $W \times d$
Energy potential	contraction and stabiliser, $-\nabla E$	$O(Tdr)$ for rank r	none
Bounded projection	hard norm bound $\|v\| \le R$	$O(Td)$	none
Gated FFN / MoE	per-token compute	$O(Td\,d_{\text{ff}})$	none

Table 2: Carried state per block. No field has a dimension that grows with the processed length T for a fixed configuration. The slot and summary accumulators preserve partial work across caller chunks; the reported equivalence result is limited to the tested configuration.

Field	Shape / content
`conv_tail`	$[B, k-1, d]$
`ssm`	$[B, Hk_{\text{exp}}, d_h, d_h]$
`memory`	keys, values, usage, write accumulators
`summary`	$[B, H, S, d_h]^{\times 2}$ + usage/pos + partial chunk
`window`	$[B, H, W, d_h]^{\times 2}$
`win_len`	scalar
`prev`	$[B, d]$ (previous normalised token)

Distributed execution uses DDP with `no_sync` on all but the final backward and an fp16/bf16 scaler protocol of unscale–clip–step–update. The schedule is warmup–stable–decay; multi-token-prediction heads are ramped from zero; the sequence length can be ramped from short windows to long ones. Cross-entropy is computed over flat token slices, never as a $[B, T, \text{vocab}]$ tensor. Full hyperparameters are in Appendix A; the defaults are interface defaults, not a claim that every reported run used them.

3.8 Implementation invariants

Five implementation invariants were found while bringing the stack from chance-level to working recall. They are documented because violating any one can silently damage the result:

1. **The delta-rule key is the previous token.** A binding is written when the value arrives at t; its key is the token at $t-1$. Reading query, key, and value all from t forces the projections to disentangle a blended pair, and they do not.
2. **The delta rule reads the clean stream, and the decay bias is protected from the generic initialiser.** A generic "initialise every bias to zero" pass resets $\alpha = \exp(-\text{softplus}(\cdot))$ to 0.5 per token, and bindings vanish in two steps.
3. **The transition is an additive residual with a convex mixture.** Replacing the residual stream with the raw latent saturates the bound and erases token identity.
4. **Operator scale is controlled before mixing.** Per-operator RMSNorm and gate-sum normalisation keep the mixture from being dominated by raw output scale or by the number of open gates; this is a stability condition, not a measured quality gain.
5. **MQAR keys must be unique per sequence.** Sampling with replacement creates ambiguous queries and caps the achievable score near 0.6.

4 Structured memory operators

4.1 The delta rule: associative recall

Per head, with $a_t = \exp(g_t)$ and $b_t = \sigma(\beta_t)$,

$$S_t = a_t S_{t-1} + b_t\, k_t \big(v_t - a_t k_t^\top S_{t-1}\big)^\top, \qquad o_t = q_t^\top S_t. \tag{3}$$

The term $(v_t - a_t k_t^\top S_{t-1})$ is error correction: the state is rewritten only by the part of the value it does not already predict. This is what makes a repeated key overwrite its binding instead of averaging all values ever seen with that key.

For training, the recurrence is evaluated in chunks of 64 tokens:

$$A[i,j] = \exp(g_i - g_j)\, b_j\, (k_i \cdot k_j), \quad j < i, \tag{4}$$

$$r = (I + A)^{-1}\big(V - \exp(g) \odot K S_0\big), \tag{5}$$

$$O = \exp(g) \odot Q S_0 + \text{tril}\big(\exp(g_i - g_j) b_j (q_i \cdot k_j)\big)\, r, \tag{6}$$

$$S_C = \exp(g_{\text{last}}) S_0 + \big(\exp(g_{\text{last}} - g_j) b_j K\big)^\top r. \tag{7}$$

Every ratio on the causal region is at most one, so the chunkwise form cannot overflow. A falsification gate

Algorithm 1: `train_worker` (code-level pseudocode).

```
INPUT: architecture A, training config C, packed token shards D
OUTPUT: latest checkpoint {model, optimizer, scaler, step, data, rng}

device    <- cuda(rank) if CUDA is available else cpu
precision <- resolve_precision(C.precision, device)
model     <- SigmaStorm(A).to(device)
optimizer <- AdamW(parameter_groups(model), lr=C.lr, betas=(.9, .95))
scaler    <- GradScaler(enabled=(precision == "fp16"))
shards    <- PackedShards(C.data_dir, rank, world_size, seed=C.seed)
step      <- load_latest_if_compatible(C.resume, model, optimizer, scaler, shards)
last_done <- step

for step in range(step, C.total_steps):
    if wall_clock_expired(C.max_hours):
        break
    optimizer.zero_grad(set_to_none=True)
    set_learning_rate(optimizer, lr_at(step, C))
    T <- seq_len_at(step, C)
    starts <- range(0, T, C.micro_chunk)
    local_tokens <- C.batch_size * T * C.grad_accum

    for accum in range(C.grad_accum):
        tokens <- shards.sample(C.batch_size, T + 1).to(device)
        inputs, targets <- tokens[:, :-1], tokens[:, 1:]
        state <- model.initial_state(C.batch_size, device)

        for window_start, window in chunks_with_index(starts, C.bptt_chunks):
            window_loss <- 0
            for index, segment in enumerate(window):
                end <- segment + C.micro_chunk
                sync <- (world_size == 1 or
                         (accum == C.grad_accum - 1 and
                          index == len(window) - 1 and
                          window_start + len(window) == len(starts)))
                with no_sync_if_needed(sync), autocast(precision, device):
                    hidden, state, extra <- model(inputs[:, segment:end], state)
                    terms <- model.loss_from_hidden(
                        hidden, targets[:, segment:end], mask=None, extra=extra)
                    loss <- weighted_loss(terms, C, local_tokens, step)
                window_loss <- window_loss + loss
            scaler.scale(window_loss).backward()
            state <- detach(state)       # truncate gradients, keep forward memory

    scaler.unscale_(optimizer)
    clip_grad_norm_(model.parameters(), C.clip)
    scaler.step(optimizer)
    scaler.update()
    last_done <- step + 1
    if is_main(rank) and save_due(last_done, C.save_every):
        save_latest(model, optimizer, scaler, last_done, shards)

if is_main(rank) and last_done > start_step and not save_due(last_done, C.save_every):
    save_latest(model, optimizer, scaler, last_done, shards)
```

Algorithm 1. Code-level pseudocode corresponding to the current `training/trainer.py` worker. A sampled sequence gets one initial state; that state is carried across all `micro_chunk` calls and detached only after each `bptt_chunks`-segment backward window. Detachment truncates the gradient graph but preserves the forward state. The weighted loss includes language modelling, configured multi-token prediction, MoE auxiliary loss, and route entropy exactly as configured; the checkpoint stores the architecture, optimiser, scaler, data cursor, and RNG state needed for resumption. Evaluation and generation use the same carried-state forward interface without the optimiser section.

compares it to the token-by-token reference and measures a maximum output difference of 4.17×10^{-7} (Table 4).

4.1.1 Decay spectrum (Gap A)

A single decay rate gives the model one timescale: it must either forget local context quickly or forget long-range bindings slowly, and it cannot do both. Each head therefore starts at its own log-spaced rate,

$$\begin{aligned} \text{rate}_h &= r_{\min}\Big(\frac{r_{\max}}{r_{\min}}\Big)^{h/(H-1)}, \\ \text{bias}_h &= \log\big(\exp(\text{rate}_h) - 1\big), \end{aligned} \tag{8}$$

with $r_{\min} = 10^{-4}$ and $r_{\max} = 3 \times 10^{-2}$, i.e. $\alpha \approx 0.9999$ down to 0.970. The projection weight is zero at initialisation, so these are exactly the initial rates, and every rate remains a learned, token-dependent function. The initialiser protects the bias from the generic zero-initialisation pass (invariant 2 in Section 3). The result is a multiscale memory at step zero: some heads can hold a binding for the whole sequence, others track the last few dozen tokens.

4.2 State expansion (Gap B)

`state_expand`$= k$ multiplies the number of independent delta states without shrinking each head:

$$\begin{aligned} &q, k, v : d \to k\,d, \\ &\quad S : [Hk,\, d_h,\, d_h], \qquad \text{output} : k\,d \to d, \end{aligned} \tag{9}$$

with $d_h = d/H$. Capacity and cost grow linearly, and unlike widening d_h this does not reduce the number of retrieval directions per token. The decay spectrum is laid out over all Hk heads, so the extra states also add timescales. The *capacity* benefit is not demonstrated at the scale tested (Section 6.3); the mechanism is implemented and the state remains constant in T.

4.3 Slot memory: gated writes and a gated read (Gap D)

A fixed ring of prototypes K_j, V_j $(j = 1 \dots n_s)$ is updated per token. Routing is a load-balanced soft assignment,

$$a_{tj} = \mathrm{softmax}_j\Big(\frac{k_t \cdot K_j - \lambda \log(1 + W_j)}{\tau_w}\Big), \tag{10}$$

where W_j is the causal write mass in the current block and $\lambda = 1$ penalises routing into an already-loaded slot. Without the penalty, sharp routing sends every similar token to the same slot and the ring collapses to one prototype.

Write gate. Both novelty and evidence gate the write:

$$g_t = \Big[\frac{\nu_t - \delta}{1 - \delta}\Big]_+ e_t, \qquad \nu_t = 1 - \max_j \cos(k_t, K_j), \tag{11}$$

where δ is the novelty floor and $[\cdot]_+ = \max(\cdot, 0)$. Novelty is computed twice: first against the consolidated prototypes, then causally against writes earlier in the same block. The second pass suppresses a repeated filler on its second occurrence instead of flooding the accumulator. The floor is a dead-zone, not a damping heuristic: below δ the write weight is exactly zero, so a repeated token cannot leak into every slot. Above the floor, the novelty term is linearly rescaled by the fixed factor $1 - \delta$ before multiplication by the evidence gate; the resulting gradient is not literally unchanged.

Consolidation. Writes accumulate for `slot_block` tokens and are consolidated with a Bayesian weight

$$D_j = \sum_t g_t a_{tj}, \quad \eta_j = \frac{D_j}{D_j + 1}, \tag{12}$$

$$\begin{aligned} V_j &\leftarrow (1 - \eta_j) V_j + \eta_j \frac{\sum_t g_t a_{tj} v_t}{D_j}, \\ K_j &\leftarrow \mathrm{normalize}\Big((1 - \eta_j) K_j \\ &\qquad + \eta_j\, \mathrm{normalize}\big(\textstyle\sum_t g_t a_{tj} k_t / D_j\big)\Big), \\ u_j &\leftarrow (1 - \eta_j) u_j + \eta_j. \end{aligned} \tag{13}$$

One confident token moves a slot slightly; a block of agreeing tokens replaces it. The accumulator, token counter, and ring are carried in the state, which is the mechanism's chunking contract. The current equivalence gate tests this behavior at a split inside a slot block; it does not measure every boundary, length, or backend.

Read gate. A softmax always hands out unit mass, so a query that matches nothing still receives a full-scale average of every slot. The first version of this operator was therefore diffuse: in a trained checkpoint the per-block read entropy was 3.03–3.16 nats against $\ln 24 = 3.178$, and zeroing the memory cost only +0.035 nats. Three changes turn the read into a gated address:

$$\begin{aligned} m_t &= \max_{j:\, u_j > 10^{-3}} \cos(q_t, K_j), \\ c_t &= \sigma\big(\kappa\,(m_t - \theta)\big), \\ \mathrm{read}_t &= c_t\, W_o \sum_j \mathrm{softmax}_j\big(s_{tj}/\tau_r\big) V_j, \end{aligned} \tag{14}$$

where $s_{tj} = q_t \cdot K_j$ is computed *before* the usage bias, and κ, θ are learned scalars. The separate read temperature τ_r is not the routing temperature: a read that is sharp before addresses are trained picks an arbitrary slot. The confidence factor is a differentiable abstention path: unmatched queries contribute zero and the residual stream alone answers them. Figure 4 shows the two paths. All three changes are opt-in (`memory_novelty_floor`, `memory_read_temperature`, `memory_abstain`); the defaults preserve the previous behaviour. Checkpoint compatibility is governed by the saved architecture configuration and should be verified for each artifact; this paper does not claim that arbitrary historical checkpoints accept every future configuration.

4.4 Chunk summaries: a bounded global path without T^2 (Gap C)

Every `summary_chunk` tokens, one pooled summary is written to a fixed ring of `summary_slots`:

$$\begin{aligned} p_c &= \frac{1}{C} \sum_{t \in c} u_t, \qquad u_t = \mathrm{RMSNorm}(x_t), \\ q_t &= \mathrm{normalize}(W_q u_t), \\ K_c &= \mathrm{normalize}(W_k p_c), \qquad V_c = W_v p_c, \\ \mathrm{score}_{tc} &= q_t \cdot K_c / \sqrt{d_h} \\ &\qquad - \mathrm{softplus}(\gamma)\,(t_c^{\mathrm{idx}} - \mathrm{pos}_c), \\ o_t &= W_o \sum_c \mathrm{softmax}_c(\mathrm{score}_{tc})\, V_c. \end{aligned} \tag{15}$$

Cost is $O(TS\,d)$ for fixed summary count, width, and chunking, and state is constant in T. The age bias is learned per head, so a summary can stay addressable for the retained ring or fade. Only completed chunks are written; the partial-chunk accumulator is carried in the state. This is a bounded global-summary path, not exact token-level retrieval for an unbounded sequence, and it avoids constructing a $T \times T$ summary attention matrix. Its measured benefit is discussed honestly in Section 6.3.

5 Experimental methodology

Artifacts. All numerical results in this report are produced by scripts in the repository: `scripts/gate.py` (falsification gates), `pytest tests` (unit tests), `scripts/ladder.py` (controlled operator comparisons),

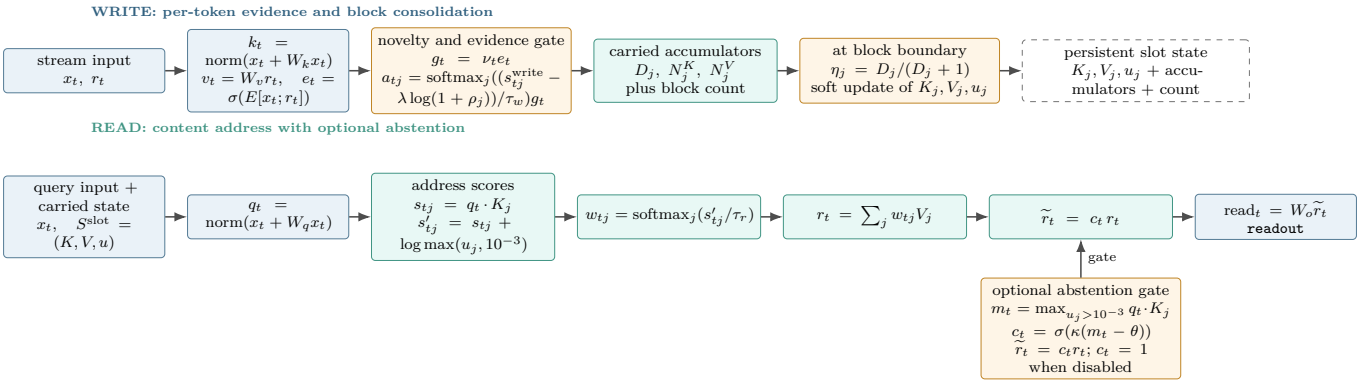


Figure 4: Slot memory implementation, separated into its two causal paths. The write path forms per-token keys, values, and evidence, computes novelty-aware assignments, accumulates them across a fixed block, and then performs the soft consolidation update. The read path uses the carried keys, values, and usage: the usage bias is added before the read softmax, while the optional match-confidence gate is computed from the raw address score and applied to the value mixture before the linear readout (equivalent to the displayed scalar form). Dashed outlines mark state that is carried across calls; the figure does not draw a shortcut from a write into the same token's read.

`scripts/gapd_proof.py` (three-arm slot-read experiment), and the training/evaluation scripts for the 21M model. Raw outputs are `results/ladder.{json,md}`, `results/gapd_proof.{json,md}`, and `results/20m_tinystories.md`.

Why synthetic tasks first. A language-model loss aggregates many capabilities and hides which operator failed. We therefore isolate one capability per task before training a language model: MQAR for delta-rule association, delayed recall by distance for the decay spectrum, a count task for the summary path, and a frozen-embedding presence task for the slot memory. The frozen-embedding detail matters: with a single supervised token per sequence, a trainable head and tied embeddings can solve the task by collapsing token embeddings without using memory. Freezing embeddings and untying the head removes that shortcut.

Tasks.

- **MQAR.** Sequences contain key→value bindings followed by queries; keys are unique per sequence. Accuracy is measured only on query answers. Chance is $1/n_{\mathrm{values}}$. The mechanism gate uses 32 keys and 16 values (chance 0.062); the ladder baseline uses 26 bindings and 16 values.
- **Delayed recall.** Eight bindings are placed at controlled distances; the model is queried later. Accuracy is reported overall and in distance buckets < 32, 32–127, 128–255, and ≥ 256 tokens.
- **Aggregate recall.** Symbols are presented, then a count question is asked about a symbol seen roughly 150 tokens earlier (max count = 8; chance 1/9).
- **Presence recall.** Sixteen symbols, eight present in the opening quarter of a 192-token stream, filler afterwards, then a query token and a yes/no answer. Only the answer is supervised. Chance is 0.50. The model is conv + slot memory only; token embeddings are frozen and the output head is untied.
- **Language modeling.** TinyStories tokenised with an 8K byte-level BPE tokenizer, EOS between documents, a 300M training-token target and a 5M held-out-token target. The checked-in validation artifact contains 4,713,394 tokens, so the realized validation size is smaller than the build target. Held-out loss is measured on that split; perplexity is exp(CE) within this tokenizer.

Protocol. Mechanism models use the `tiny` preset; task-specific overrides are listed in Appendix A. Unless stated otherwise: AdamW with learning rate 10^{-3}, betas $(0.9, 0.95)$, weight decay 0.01, gradient clipping at norm 1.0. Batch sizes are 8–24 depending on the task; the presence experiment uses batch 8 and 800 steps, the aggregate comparison 600 steps, and MQAR 400 steps. Seeds are reported individually because the seed-to-seed spread is a first-class result, not a nuisance to be hidden by a mean. With two or three seeds we do not compute significance tests; a difference smaller than the seed spread is called inconclusive.

Metrics. Accuracy is the fraction of supervised answer tokens that are correct. Cross-entropy is reported in nats per supervised token, CE = lm_sum/lm_count, and perplexity as exp(CE). Read entropy is $H = -\sum_j p_j \log p_j$ averaged over tokens, with $\ln n_{\mathrm{slots}}$ the uniform maximum. The memory ablation delta is $\Delta_{\mathrm{abl}} = \mathrm{CE}(\text{memory output zeroed}) - \mathrm{CE}(\text{normal})$; a positive value means the memory is useful. State bytes are the sum of all carried tensors per sample, measured at two context lengths to verify independence from T.

Hardware and honesty about it. The 21M language-model run used two 16 GB T4 GPUs (fp16 DDP, 13.49 GB peak per GPU). The mechanism and Gap D experiments in this report were re-run on CPU because the Kaggle weekly GPU quota (30 h) was exhausted; data, seeds, optimiser, and step counts are identical to the GPU protocol. The forward-time measurements are single-machine CPU numbers with two threads and should be read as relative cost, not deployment throughput.

Table 3: Claims and evidence map. "Established" means a controlled comparison or an equivalence gate with a stated number; "inconclusive" means the observed difference is within seed/task noise; "not demonstrated" means the mechanism exists but its benefit was not shown. No row in this table is a frontier-scale claim.

Claim	Evidence	Status
Chunkwise delta rule equals the sequential rule	gate: max output diff 4.17×10^{-7}	Established
Chunked forward equals full forward	gate: 1.6×10^{-6}, split inside slot/summary blocks	Established
State size is independent of context	200,704 bytes at $T = 512$ and $T = 4096$	Established
Decay spectrum improves long-range recall	≥ 256: 1.000 vs. 0.297 (frozen rates)	Established (synthetic)
State expansion increases capacity	expand1 0.754 vs. expand2 0.555 at 66 bindings	Not demonstrated
Chunk summaries improve aggregate recall	0.703 with vs. 0.672 without; an earlier 0.797 vs. 0.891 did not reproduce	Inconclusive
Gated slot read is content-addressed	presence 0.977/1.000/0.953 vs. chance; $\Delta_{\text{abl}} = +2.26$ nats	Established (synthetic)
Gated read leaves the checked controls close	MQAR is higher in both seeds; aggregate is mixed; one LM pair is 3.0826 vs. 3.0760 nats	Mixed; no improvement claim
Small language model trains and generates	21M params, 348M tokens, held-out loss 1.2523 / PPL 3.498	Prototype demonstrated
Competitive with transformers at LM	no matched LM baseline was trained	Not claimed
256K-token context works	no model trained at that context	Not demonstrated

Table 4: Falsification gates (`scripts/gate.py`) and unit tests.

Gate	Measured
Delta rule, chunkwise vs. sequential	$\max \lvert\Delta o\rvert = 4.17 \times 10^{-7}$
	$\max \lvert\Delta S\rvert = 3.58 \times 10^{-7}$
Chunked vs. full forward (split at 50)	$\max \lvert\Delta h\rvert = 1.55 \times 10^{-6}$
State bytes at $T = 512$ and $T = 4096$	200,704 (constant)
MQAR recall, 400 steps	0.974 vs. chance 0.062
Unit tests (`pytest tests`)	21 passed

Table 5: Delayed recall by distance (8 bindings, length 320, 300 steps). Frozen decay.

Variant	Overall	< 32	32–127	128–255	≥ 256
full	0.984	0.984	0.984	1.000	0.969
spectrum	**1.000**	1.000	1.000	1.000	**1.000**
single rate	0.758	0.984	1.000	0.750	0.297

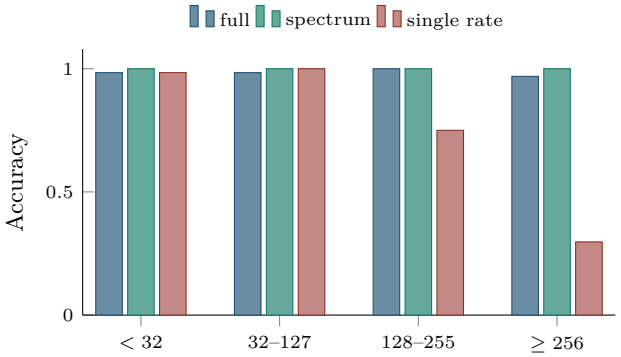


Figure 5: Recall collapses with distance when all heads share one frozen decay rate. The log-spaced spectrum holds every bucket; the full model is slightly below the delta-only spectrum at the longest bucket because the comparison mixes additional operators.

6 Results

6.1 Correctness gates

Table 4 lists the falsification gates. They are not capability results; they establish that the implementation matches its mathematical specification and that the state contract holds. The chunked-forward gate is the important one for long context: the split point is position 50, inside both a 64-token slot-consolidation block and a 32-token summary block, and the chunked hidden state still matches the full forward to 1.6×10^{-6}.

6.2 Decay spectrum (Gap A)

Table 5 and Figure 5 report delayed recall at four distance buckets. The comparison is deliberately isolated: the slot memory and summaries are disabled, and the decay bias is frozen so the comparison measures the initialised prior. When the bias is trainable, a single-rate model simply learns its own spectrum and the ablation shows nothing; that was the first, invalid version of this experiment (Appendix C).

The single rate is not broken at short distances: it reaches 0.984 below 32 tokens and 1.000 at 32–127. The failure is specifically long-range, 0.297 beyond 256 tokens, against 1.000 for the spectrum. A single timescale cannot simultaneously forget local context and preserve a binding across hundreds of tokens; the spectrum can.

6.3 State expansion and chunk summaries (Gaps B and C)

Table 6 reports the two comparisons whose benefit is *not* established.

Table 6: Gap B (dense MQAR, 66 bindings) and Gap C (aggregate count). Both tasks are seed-noise-limited; see the text.

Variant	Accuracy	Params
Capacity: MQAR, 66 bindings (chance 0.125*)*		
`state_expand`=1	**0.754**	1,694,911
`state_expand`=2	0.555	2,027,231
Aggregate count (chance 0.111*)*		
full (with summaries)	0.703	2,080,744
no summaries	0.672	1,817,556

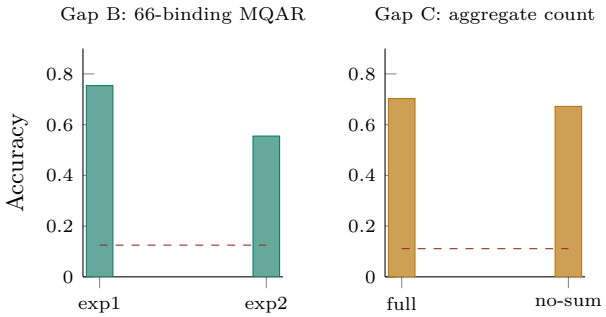


Figure 6: Neither extra state capacity nor the summary ring shows a reliable benefit at the tested budget. Dashed lines are chance. These are negative/inconclusive results, not hidden failures.

State expansion. Doubling the number of independent delta states did not improve dense MQAR; it reduced accuracy from 0.754 to 0.555. The task has 66 bindings, above the practical rank of one 32×32 state per head, so one might expect capacity to help. It did not at 300 steps. Possible explanations are optimisation (the larger model needs more steps) or interference between duplicated state matrices; the experiment does not distinguish them. We therefore do not claim a capacity benefit.

Chunk summaries. The aggregate count task improves by 0.031 with summaries, 0.703 vs. 0.672. An earlier run measured 0.797 with summaries and 0.891 without, i.e.the opposite sign. The metric is noise-limited at this budget, and the summary change was not adopted on the strength of either run. The operator is correct and chunk-invariant; its quality benefit remains unproven.

6.4 The gated slot read (Gap D)

Table 7 reports the three-arm presence experiment: the previous cosine read, the ZCA-whitened read on its own, and the shipped gated mechanism (novelty dead-zone, read temperature, abstention). All arms use identical data, seeds, optimiser, and 800 steps.

The gated read is not merely sharper; it is discriminating. Match confidence averages 0.562 on present queries and 0.026 on absent ones, i.e.the abstention path separates the two populations without supervision on the gate. The ablation delta shows the same from the other side: zeroing the memory now costs 2.26 nats, against 0.15 before, so the residual stream alone can no longer answer the task. ZCA whitening sharpens entropy from 1.26 to 0.72 nats but leaves accuracy at chance, because it amplifies every deviation direction, including context noise.

Table 7: Presence recall with frozen embeddings (16 symbols, 8 present, chance 0.50). Means over seeds are shown for readability; individual seeds are the evidence.

Arm	Seed 0	Seed 1	Seed 2	Mean
cosine (previous)	0.562	0.539	0.555	0.552
ZCA-whitened	0.648	0.531	0.539	0.573
gated	**0.977**	**1.000**	**0.953**	**0.977**

Table 8: Gap D diagnostics, seed 0, 16 slots. Uniform read entropy is $\ln 16 = 2.773$ nats.

Arm	Entropy	Top w	Key cos	CE	Δ_{abl}
cosine	1.260	0.608	0.023	0.663	+0.147
whitened	0.725	0.788	−0.044	0.517	+0.217
gated	**0.413**	**0.820**	0.004	**0.019**	**+2.255**

No-regression controls. The gated mechanism must not break the paths it does not target. Table 9 shows MQAR (delta-rule path), aggregate recall (summary path), and a 600-step real-data next-token comparison on the tiny preset. MQAR is slightly higher for the gated arm (+0.002 and +0.008), but this is only two seeds. Aggregate recall is mixed: the gated arm is higher on seed 0 (0.836 vs. 0.680) and lower on seed 1 (0.859 vs. 1.000). The single paired language-model check moves validation CE from 3.0760 to 3.0826 nats; without repeated runs, this does not establish a regression or an improvement. The plotted training curves are similar over the recorded steps (Figure 8).

6.5 Language modeling at 21M parameters

The largest model in this report has 20,996,090 parameters (6 blocks, width 320, 8 heads, `state_expand`=2, 24 slots, 32 summary slots, no MoE). It was trained on TinyStories (8K byte-level BPE, 300M-token training split) for 10,604 steps in two resumed sessions on two T4 GPUs, a total of roughly 348M tokens. Table 10 reports the two held-out evaluations.

The model writes grammatical, on-genre TinyStories prose from a prompt. The checkpoint predates the gated slot read; its slot memory is the diffuse version audited in Table 11, which is exactly the defect Gap D removes. The language model is useful as a prototype, not as evidence of general language ability: TinyStories is a restricted synthetic corpus, the tokenizer is 8K byte-level BPE, and no transformer was trained on the same data for comparison.

6.6 Cost and state

Table 13 compares SigmaStorm with the size-matched-in-width transformer control from the experiment ladder. The control matches width, depth, heads, and FFN multiplier, not total parameter count: SigmaStorm's operators add parameters, so at the same width it has about 2.3–3.0× more parameters and, on CPU, is roughly 2–9×

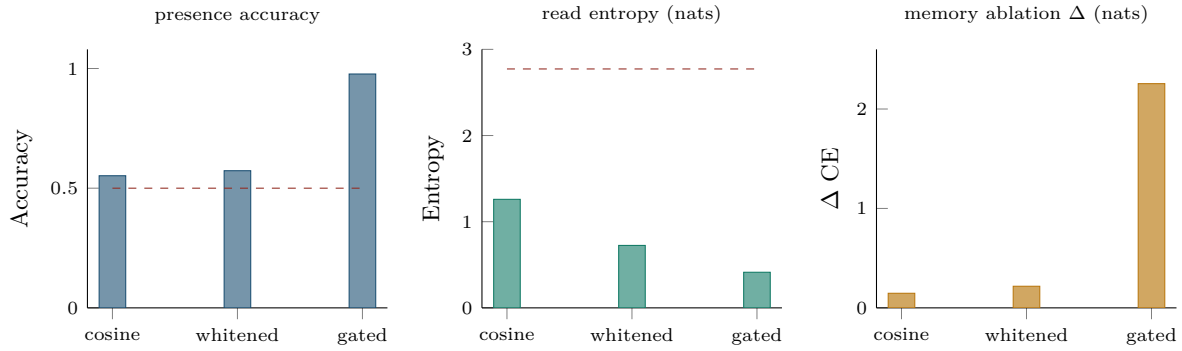


Figure 7: Gap D in one view. Whitening sharpens the read (entropy falls) but does not make it useful; the gated read both sharpens it and makes it load-bearing, as shown by the ablation delta. Dashed lines: chance accuracy, uniform entropy $\ln 16 = 2.773$ nats.

Table 9: No-regression controls for the gated read. MQAR: 400 steps, 2 seeds. Aggregate: 600 steps, 2 seeds. LM: tiny preset, 3.06M params, 600 steps, identical stream.

Measure	previous	gated
MQAR seed 0	0.988	**0.990**
MQAR seed 1	0.979	**0.987**
Aggregate seed 0	0.680	**0.836**
Aggregate seed 1	1.000	0.859
LM val CE (nats)	**3.0760**	3.0826
LM PPL	21.67	21.82

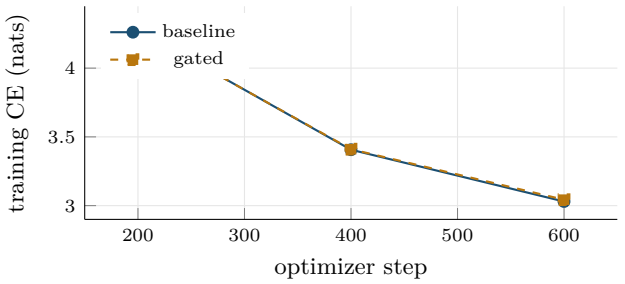


Figure 8: Real TinyStories next-token training on an identical stream. The two recorded loss trajectories are similar over 600 steps; this single paired run does not provide uncertainty. Held-out CE is 3.0760 (baseline) vs. 3.0826 (gated).

slower per forward pass. There is no compute-efficiency claim here. The architectural advantage that motivates the design is memory: state is constant in context (Table 4), while a transformer KV cache grows linearly. That advantage only matters if long-context training works at scale, which is not demonstrated in this report.

7 Discussion

What the evidence supports. Three claims survive the controls. First, the transition law is executable and its chunked implementation matches the sequential reference and the full forward to numerical precision, including at boundaries that cut through consolidation and summary blocks. Second, a multiscale decay initialisation is not cosmetic: with the bias frozen, a single rate loses long-range recall while the spectrum retains it. Third, a memory read needs an abstention path; a sharper softmax alone does not make a store content-addressed.

Table 10: 21M TinyStories run, 2 × T4, fp16 DDP. Held-out evaluation is 20 batches of 4 × 1024 tokens (81,840 supervised tokens). MQAR is far out of distribution and is reported for completeness only.

Session	Steps	Held-out loss	PPL
1	5,442	1.3225	3.753
2	10,604	**1.2523**	**3.498**

Table 11: Slot-memory audit of the 21M checkpoint (prefix). Uniform read entropy is $\ln 24 = 3.178$ nats. Zeroing the read costs only +0.035 nats: the memory acts as a slow average. This table motivates the gated read.

Block	Read entropy	Top weight	Cross-slot cos
0	3.133	0.065	0.465
1	3.109	0.073	0.525
2	3.160	0.070	0.810
3	3.159	0.055	0.715
4	3.048	0.131	0.716
5	3.095	0.078	0.483

Hypotheses and evidence levels. The following map separates mechanisms that are implemented from hypotheses that have actually survived a controlled test.

Sharpness is not addressing. The Gap D result is the clearest lesson of the study. ZCA whitening reduced read entropy from 1.26 to 0.72 nats and still left presence recall at chance. Entropy measures concentration, not correctness: whitening amplifies every deviation direction, including the context noise that should not be retrieved. The gated read adds two things that entropy does not capture: a write path that refuses redundancy, and a read path that can return *nothing*. The confidence factor separates present (0.56) from absent (0.03) queries without direct supervision, and the ablation delta rises by an order of magnitude. A memory operator without a null

Table 12: Representative samples from the step-10,604 checkpoint (temperature 0.7, top-k 40), truncated. These show local coherence, not long-horizon recall.

"Once upon a time, there was a little girl named Lily. She loved to play with her toys and make them into different things. One day, she found a toy that was different from the others. It was a big, shiny rock that had a pointy edge. . . "
"The little dog ran into the garden and saw the flowers. He was so happy and he couldn't wait to show all his friends. But then, something bad happened. A big dog came running and scared the little dog away. . . "
"Tom was very sad because he lost his toy. He looked everywhere for it, but it was nowhere to be found. He started to cry because he missed his toy so much. His mother was worried and asked him what was wrong. . . "

Table 13: Forward cost at equal width (CPU, two threads, batch 1). The transformer control matches width/depth, not parameter count.

Config	Σ params	Σ s	T params	T s
tiny T=128	2.08M	0.031	0.68M	0.0036
tiny T=512	2.08M	0.102	0.68M	0.0147
tiny T=2048	2.08M	0.411	0.68M	0.1404
20M T=128	20.97M	0.094	8.37M	0.0180
20M T=512	20.97M	0.321	8.37M	0.0761
20M T=2048	20.97M	1.339	8.37M	0.5872

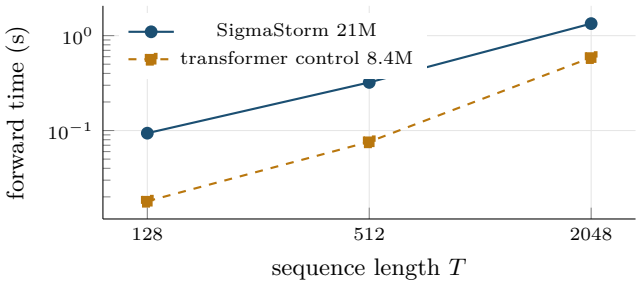


Figure 9: Forward cost of the 21M configuration and the width-matched transformer control on CPU. The three tested points are approximately linear on this log–log plot, but this does not establish an asymptotic guarantee for the dense implementation. SigmaStorm's operators cost more per token in this measurement; the relative gap is an engineering problem, not a property of the architecture alone.

hypothesis cannot express "this is not in memory", and no amount of temperature tuning supplies one.

Negative results are part of the design. State expansion was intended to add capacity; at 66 bindings it made accuracy worse. Chunk summaries were intended to add a global path; their measured benefit is within noise and an earlier run had the opposite sign. Reporting these results changes the architecture's story: the demonstrated recall mechanisms are the delta rule and the gated slot read, not the extra state matrices or the summary ring. The summary ring remains valuable as a design element—it is correct, chunk-invariant, and asymptotically cheaper than attention—but its quality benefit is an open question, not a result.

Why there is no transformer language-model comparison. The experiment ladder's transformer is a width-matched control for synthetic recall, not a language model. It was never trained on TinyStories. A matched LM comparison requires the same tokenizer, corpus, token budget, optimiser, evaluation split, and roughly matched parameters or FLOPs; that run has not been performed. Without it, statements such as "SigmaStorm is better or worse than a transformer" are unsupported. This is the single most important missing experiment, and it is cheap relative to the scale debate: a 21M transformer on 348M TinyStories tokens fits the same two-T4 budget.

The long-context claim is architectural, not empirical. The state contract is real: 200,704 bytes at $T = 512$ and $T = 4096$ for the tiny preset. The reported split test supports the carried-state implementation for one configuration, but does not prove every operator and boundary on every backend. Constant configured state is a prerequisite for streaming a 256K-token sequence in bounded state memory, not evidence that the model can usefully model 256K tokens. The 21M run used 1,024-token windows with a differentiable horizon of 4×256 tokens in the reported configuration. Long-context ability—retrieval over tens of thousands of tokens, stability of the energy-controlled latent, and the interaction of the summary ring with the delta state—has not been measured. The design target is 256K; the evidence stops far short of it.

Engineering cost. At equal width SigmaStorm is slower than the transformer control on CPU and has more parameters. The operators are written in dense PyTorch; there are no fused kernels for the chunkwise delta solve, the slot consolidation, or the summary scan. The gap in Figure 9 is therefore an implementation gap as much as an architectural one. A fair efficiency comparison would use fused kernels and measure tokens per second at matched parameter count and matched training budget. That work is not part of this report.

What would change the conclusion. Four experiments, in order of value: (i) a matched transformer LM baseline on the same corpus and budget; (ii) a gated-read 21M run to confirm that the synthetic gain survives language modeling; (iii) a capacity-stressed state-expansion run with enough steps for both variants to converge; (iv) a summary ablation with multiple seeds and a longer budget. If (i) shows a large deficit, the architecture needs a different argument than quality-per-token. If (ii) shows no change, the slot memory is not used by the language model even when it can abstain, which is itself an important negative result.

Table 14: Hypotheses, evidence, and current status.

Hypothesis	Evidence in this report	Status
Carried state and caller-independent forward behavior	One split inside a slot and summary block; max hidden difference 1.6×10^{-6}	Demonstrated for the tested configuration
Multiple decay rates and delayed recall	Recall at distances ≥ 256: 1.000 vs. 0.297	Supported on the frozen synthetic task
An abstaining slot read is more useful than a sharper always-on read	Gated presence recall 0.977/1.000/0.953; whitening remains near chance	Supported on the frozen-embedding task
More delta states increase usable capacity	Expand 1: 0.754; expand 2: 0.555 at 66 bindings	Not supported by the reported run
Chunk summaries improve aggregate quality	0.703 vs. 0.672 in one run; earlier run had the opposite sign	Inconclusive
The design supports 256K-token modeling	256K is a design target; no training or evaluation at that length	Not tested

8 Limitations

The results are deliberately narrow. This section lists what the study cannot support, so that the claims in Table 3 are not read more broadly than the evidence allows.

Statistical power. Most comparisons use two or three seeds, and we do not compute significance tests. The seed-to-seed spread on the aggregate task is larger than the effect being measured, which is why Section 6.3 calls it inconclusive rather than positive. A single seed of the largest Gap D arm would not have been convincing; the three-seed presence result is credible because every seed is far above chance and the arms are otherwise identical. The Gap A result is a controlled ablation with frozen parameters, so seed noise is small, but it remains a single task.

Construct validity of the synthetic tasks. MQAR, delayed recall, aggregate count, and presence recall each isolate one operator. That is their purpose, but isolation is also their weakness: a mechanism that works on a synthetic binding task need not be the mechanism a language model uses. The presence task in particular uses frozen embeddings and untied heads to remove a shortcut; that control changes the optimisation problem relative to a normal language model, so the 0.977 accuracy should be read as evidence about the read operator, not as a prediction of LM behaviour.

No matched transformer language-model baseline. This is the most important omission. The transformer in the experiment ladder is a width-matched control for synthetic recall and was never trained on TinyStories. It therefore cannot answer whether SigmaStorm is better or worse than a transformer at language modeling. Such a comparison requires the same tokenizer, corpus, token budget, optimiser, evaluation split, and roughly matched parameters or FLOPs. Until it exists, no capability comparison with transformers is supported, including comparisons implied by the architecture's motivation.

Scale. The largest model is 21M parameters, trained for roughly 348M tokens on TinyStories. There are no scaling curves, no runs at 100M or 1B parameters, and no data-mix study. TinyStories is a restricted corpus with simple syntax; success there says little about natural text. The tokenizer is an 8K byte-level BPE, so the reported perplexity is not comparable to perplexities computed with larger subword vocabularies.

Long context. The carried-state implementation is designed to be independent of caller chunk boundaries, and the state is constant in T for a fixed configuration; the reported equivalence gate covers one split only. The largest trained run used 1,024-token windows with a differentiable horizon of 4×256 tokens in the reported configuration. The 256K design target has not been trained or evaluated. Specifically unmeasured: retrieval over tens of thousands of tokens, stability of the energy-controlled latent over long streams, the interaction of the summary ring and the delta state at scale, and the quality of generation after chunked prefill. The chunking result is an implementation check, not a learned long-context capability result.

Efficiency measurements. The cost table reports CPU forward times with two threads and batch size one. Those numbers measure dense PyTorch implementations without fused kernels for the chunkwise delta solve, the slot consolidation, or the summary scan. They are useful as a relative sanity check over the tested lengths; they do not establish asymptotic runtime for every operator, deployment throughput, or a fair efficiency comparison against optimized attention kernels. In particular, the current window-attention path materialises a mask over each input chunk.

Hardware and reruns. The 21M language-model run used two 16 GB T4 GPUs. The mechanism and Gap D experiments were re-run on CPU after the weekly GPU quota was exhausted, with identical data, seeds, optimiser, and step counts. This is honest but not ideal: CPU and GPU floating-point behaviour differ, and the re-runs do not exercise the distributed training path. The absence of a multi-GPU scaling result is a gap in the engineering

evidence, not just in the science.

Evaluation breadth. There are no downstream benchmarks, no zero-shot evaluations, no instruction tuning, and no human evaluation. The 21M model is a prototype that produces the recorded on-genre samples; it is not evaluated as an assistant. Claims about usefulness, safety, or alignment are outside the scope of this report and are not made.

Post-training and fine-tuning scope. The repository currently supports language-model training, checkpoint resume, evaluation, and generation. It does not implement instruction supervised fine-tuning, chat-template handling, assistant-span loss masking, preference optimisation such as DPO or RLHF, LoRA/adapters, quantisation, serving, or a safety-evaluation suite. Those are future protocols, not hidden capabilities of the current checkpoint. A valid follow-up would define the data format and special tokens, start from a checkpoint with its saved architecture configuration, reserve a held-out instruction set, compare pre- and post-tuning loss and task scores, and report safety and regression evaluations separately.

Known negative and unstable results. State expansion reduced accuracy at 66 bindings, and the summary benefit changed sign between runs. Both are reported as results rather than suppressed. They limit the architectural story: the demonstrated recall mechanisms are the delta rule, the decay spectrum, and the gated slot read, not the additional state matrices or the summary ring. A reader should not infer that every component of the block contributes measured value.

Reproducibility caveats. All headline numbers are produced by scripts in the repository, and the raw artifacts are checked in. Reproducing them still requires the same package versions and, for the 21M run, the same two-GPU configuration. We report seeds and step counts, but we do not claim bitwise determinism across hardware. The paper's evidence policy is traceability to an artifact, not independent replication.

9 Conclusion

SigmaStorm is an attempt to replace dense token-to-token attention with a bounded recurrent state whose transition law is explicit. The implementation is small, the evidence is small, and the report is written to make the boundary between the two visible.

The controlled results are narrow but reproducible at prototype scale. The chunkwise delta rule matches its sequential reference to 4.2×10^{-7}, and a chunked forward matches the full forward to 1.6×10^{-6} for the tested split inside a slot-consolidation and a summary block. The state contract is real: 200,704 bytes at $T = 512$ and $T = 4096$. A log-spaced decay spectrum retains long-range recall in the frozen-rate task, 1.000 vs. 0.297 beyond 256 tokens. A gated slot read makes the memory useful on the frozen-embedding presence task: recall is 0.977/1.000/0.953 over three seeds and the memory-ablation delta is +2.26 nats. The ancillary controls are mixed: MQAR is slightly higher in both tested seeds, aggregate recall is higher on one seed and lower on the other, and one paired LM check has CE 3.0826 vs. 3.0760 nats.

Two results are not established. Doubling the number of delta states reduced MQAR accuracy at 66 bindings, and the summary ring's benefit is within noise and changed sign between runs. The architecture's demonstrated recall mechanisms are therefore narrower than the full block: the delta rule, the decay spectrum, and the gated slot read carry the evidence, while state expansion and chunk summaries remain implemented and correct but empirically unproven.

The largest model, 21M parameters on 348M TinyStories tokens, reaches recorded held-out loss 1.2523 and PPL 3.498 and produces locally coherent on-genre samples. It is a prototype, not a language-model comparison. There is no matched transformer baseline, no 256K training, and no downstream evaluation. The architectural claim that motivates the design—fixed configured state capacity—is a property of the state contract, not a measured long-context capability.

What would change the conclusion is clear. A matched transformer LM baseline on the same corpus and budget would settle the quality question. A gated-read 21M run would show whether the synthetic gain survives language modeling. A capacity-stressed state-expansion run and a multi-seed summary ablation would resolve the two open mechanisms. These are cheap relative to the scale debate and should precede any larger training effort.

The honest summary is this: the mechanisms are executable, the correctness gates hold, and one memory defect was found, diagnosed, and fixed with a controlled experiment. That is a foundation for further work, not a frontier result. The report is written so that a reader can check every number and see exactly where the evidence stops.

References

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Kimi Team. Kimi K3: Open frontier intelligence. *arXiv preprint arXiv:2607.24653*, 2026.

DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. *arXiv preprint arXiv:2609.19969*, 2026.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. In *arXiv preprint arXiv:2004.05150*, 2020.

Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, et al. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, 2020.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, 2021.

Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *Transactions on Machine Learning Research*, 2024. arXiv:2312.00752.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, et al. RWKV: Reinventing RNNs for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *International Conference on Machine Learning*, 2024. arXiv:2312.06635.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025. arXiv:2412.06464.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

Hubert Ramsauer, Bernhard Schäfl, Johannes Lehner, Philipp Seidl, Michael Widrich, Lukas Gruber, Markus Holzleitner, Thomas Adler, David Kreil, Sepp Hochreiter, and Günter Klambauer. Hopfield networks is all you need. In *International Conference on Learning Representations*, 2021.

Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. *arXiv preprint arXiv:2312.04927*, 2023.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.

A Configurations

All experiments use `SigmaConfig` from `sigmastorm/config.py`. Unknown fields are rejected at load time, so a checkpoint cannot silently acquire new behaviour. Table 15 lists the two presets used in this report; values not shown take their dataclass defaults (Table 16). Task-specific overrides are limited to batch size, step count, sequence length, and the operator switches named in the text.

Table 15: Presets used in this report. The 21M language-model run uses the `20m` preset with `state_expand`=2, 24 slots, and 32 summary slots.

Field	`tiny`	`20m`	Role
`vocab_size`	512	8,192	byte-level BPE (8K) for TinyStories
`d_model`	128	320	residual width
`n_blocks`	4	6	depth
`n_heads`	4	8	delta-rule heads before expansion
`slot_count`	16	24	prototype ring size
`window`	64	128	window-attention length
`state_expand`	2	2	independent delta states per head
`summary_chunk`	32	128	tokens per pooled summary
`summary_slots`	16	32	summary ring size

Table 16: Selected defaults shared by both presets.

Field	Value	Note
`chunk_size`	64	chunkwise delta solve
`conv_kernel`	4	short convolution width
`slot_block`	64	consolidation block
`slot_temperature`	0.5	routing temperature
`local_attn_every`	4	one window block in four
`summary_every`	2	one summary block in two
`ffn_mult`	2.67	FFN expansion
`max_norm` R	16	projection radius
`dt`	0.5	base step size
`energy_rank`	32	low-rank energy factor
`energy_floor`	10^{-2}	diagonal energy floor
`decay_min_rate`	10^{-4}	slowest initial rate
`decay_max_rate`	3×10^{-2}	fastest initial rate
`mtp_horizons`	$(2, 4)$	multi-token prediction offsets
`loss_token_chunk`	256	chunked LM-loss slice
`tie_embeddings`	true	input/output embedding tie
`dropout`	0.0	no dropout in reported runs

The Gap D switches are opt-in and default to the previous behaviour: `memory_novelty_floor` = 0, `memory_read_temperature` = `None`, and `memory_abstain` = false. The shipped gated configuration sets a positive novelty floor, a read temperature separate from the routing temperature, and `memory_abstain` = true. The defaults reproduce the old operator, but checkpoint compatibility should be checked against each saved architecture configuration rather than assumed for arbitrary historical artifacts.

B Reproduction

All commands run from the repository root with the project virtual environment (`.venv/bin/python`, CPU build is sufficient for the mechanism experiments). The falsification gates and unit tests are the first check and should pass before any result is cited.

```
.venv/bin/python scripts/gate.py
.venv/bin/python -m pytest -q tests
.venv/bin/python scripts/ladder.py \
    --out_dir results
.venv/bin/python scripts/gapd_proof.py \
    --out_dir results
```

The Gap D proof script runs the three arms (cosine, ZCA-whitened, gated) on the presence task for seeds 0–2 at 800 steps, then the MQAR and aggregate no-regression controls. The 21M language-model run is reproduced by the training script with the `20m` preset and TinyStories; it requires two 16 GB T4 GPUs in the reported configuration. Raw artifacts: `results/ladder.{json,md}`, `results/gapd_proof.{json,md}`, and `results/20m_tinystories.md`.

C Honesty ledger

This appendix records experiments that failed, were invalidated, or changed sign. They are part of the evidence: a report that only lists what worked is not a record of what was learned.

Table 17: Honesty ledger. "Invalidated" means the first version of the experiment did not measure what it claimed; the corrected version is the one reported in the body.

Experiment	What happened	Status in body
Gap A, first version	Decay bias left trainable. A single-rate model learned its own spectrum and the ablation showed no difference. The comparison measured training, not the initialised prior.	Invalidated; frozen-bias version reported
Gap B, state expansion	Intended to add capacity; at 66 bindings it reduced accuracy from 0.754 to 0.555. Optimisation and interference are not separated.	Negative result reported
Gap C, summaries	First run: 0.797 with summaries vs. 0.891 without, i.e. the opposite sign. Second run: 0.703 vs. 0.672. Both are within seed noise.	Inconclusive; not adopted
Gap D, whitening alone	ZCA whitening cut read entropy from 1.26 to 0.72 nats but left presence recall at chance (0.573 mean). Sharpness is not addressing.	Negative result; motivated the gate
Gap D, first slot audit	In the 21M checkpoint the read entropy was 3.03–3.16 against $\ln 24 = 3.178$, and zeroing the memory cost +0.035 nats: the memory was a slow average.	Defect reported; fixed by the gated read
21M checkpoint	Predates the gated slot read. Its slot memory is the diffuse version, so the LM run does not demonstrate the fixed mechanism.	Stated explicitly
Hardware	Weekly Kaggle GPU quota exhausted; mechanism and Gap D runs re-executed on CPU with identical protocol.	Stated in Section 5
MQAR keys	Early runs sampled keys with replacement, making queries ambiguous and capping accuracy near 0.6. Keys are now unique per sequence.	Implementation invariant (I4)
Decay bias init	A generic zero-bias initialiser reset every decay to $\alpha = 0.5$ and bindings vanished in two steps. The spectrum initialiser now protects those parameters.	Implementation invariant (I2)

What is deliberately not in this report. No result on 256K-token training, no matched transformer language-model baseline, no downstream benchmark scores, and no multi-GPU scaling measurement. These are not omissions by accident: the runs were not performed, and the corresponding rows of Table 3 say so. Where a number is a prototype artifact (the 21M held-out loss, the synthetic accuracies) the text says prototype; where a property is architectural (constant state, chunk invariance) the text says architectural. The distinction is the point of the report.